Graded Lab #1 (VERSION.1)

AI (E11)

Artificial Intelligence – Lab | University of Central Punjab

Time: 1 hour	Total Marks: 25	Language: Python
Name: _____	Reg. No: _____	Date: _____

Plagiarism Policy

- **This is an individual assessment. All code must be written by you during the lab session.**
- **Copying code from another student, sharing your code, or allowing others to copy it is plagiarism and is treated as cheating by both parties.**
- **Using AI tools, online solutions, websites, messaging apps, or unauthorised notes to generate or obtain answers is not allowed.**
- **Code that is identical or suspiciously similar (including identical variable names, comments or mistakes) will be treated as plagiarism.**
- **You may be asked to explain your code. If you cannot explain it, you will receive zero marks for that question.**
- **Any student found guilty will receive zero marks in this graded lab, and the case will be reported according to university rules.**

Question 1: Compress Repeated Characters (10 marks)

Task

Write a function **compress(text)** that returns a **list** in which every block of identical neighbouring characters is replaced by a single copy of that character.

Rules

- Only characters that are next to each other are collapsed. The same character appearing again later in the string must be kept.
- Case matters: 'A' and 'a' are different characters.
- **Do not use set(), dict, or the in operator on the result list to check for repeats.**
- The function must return the list, not just print it.
- **It must run in O(n).** Use a single loop and compare each character only with the one before it.

Hint

- Use len(text) to get the number of characters, so you know how long the loop should run.
- Characters are accessed by index: text[0] is the first character.
- Start with an empty list result = [].
- Handle the first character separately, since it has no previous character to compare with.
- Check for an empty string before reading text[0], otherwise the program will crash.

```
string1 = "aabbbcca"
print(len(string1))   # prints 8
```

Samples

Sample 1

```
Input:  "aaabccdd"
Output: ['a', 'b', 'c', 'd']
```

Sample 2

```
Input:  "aabbbcca"
Output: ['a', 'b', 'c', 'a']
```

(The final 'a' is kept because it is not next to the first block of 'a'.)

Input	Expected output
"aaabccdd"	['a', 'b', 'c', 'd']
"aabbbcca"	['a', 'b', 'c', 'a']
"abab"	['a', 'b', 'a', 'b']
""	[]

Question 2: Bank Account System (15 marks)

Task

Ask the user for an opening balance and store it in a variable **balance**. Then show a menu in a **while** loop until the user chooses Exit:

```
1. Deposit
2. Withdraw
3. Show balance
4. Withdrawals possible
5. Exit
```

Option	What it does	Marks
`1. Deposit`	`Uses deposit(balance, amount)`	`1.25`
`2. Withdraw`	`Uses withdraw(balance, amount)`	`1.25`
`3. Show balance`	`Prints the current balance`	`1.25`
`4. Withdrawals possible`	`Uses withdrawals_possible(balance, amount)`	`10`
`5. Exit`	`Prints a goodbye message and leaves the loop`	`1.25`

Functions

Do not use classes or global variables.

- **deposit(balance, amount)** returns the new balance after adding amount. If amount is zero or negative, it returns balance unchanged.
- **withdraw(balance, amount)** returns the new balance after subtracting amount. If amount is zero, negative, or greater than balance, it returns balance unchanged.
- **withdrawals_possible(balance, amount)** uses loop to count how many times amount can be withdrawn from balance before the balance is too low for another withdrawal. It returns the count. If amount is zero or negative, it returns -1.

Call	Expected output
`deposit(500, 200)`	`700`
`deposit(500, -50)`	`500`
`withdraw(500, 200)`	`300`
`withdraw(500, 800)`	`500`
`withdraw(500, 0)`	`500`
`withdrawals_possible(1000, 300)`	`3`
`withdrawals_possible(1500, 400)`	`3`
`withdrawals_possible(200, 300)`	`0`
`withdrawals_possible(1000, 0)`	Enter Valid amount!!

Rules

- The balance variable must always hold the current balance. After every deposit or withdrawal, store the returned value back into balance (for example, balance = deposit(balance, amount)), so the next menu choice uses the updated amount.
- Option 3 must show the current balance, and option 4 must use the current balance.
- Option 4 only reads the balance. It must not change balance.
- Your program must defend against errors and crashes caused by invalid input.
- If a deposit or withdrawal fails, print a clear message.

Hint

- input() always returns a string, so convert it with int() or float().
- Use break to leave the loop when the user chooses Exit.

Sample run

```
Enter opening balance: 1000
Choose option: 1
Enter amount: 500
Balance: 1500
Choose option: 2
Enter amount: 2000
Withdrawal failed: invalid amount or insufficient balance
Choose option: 2
Enter amount: 300
Balance: 1200
Choose option: abc
Invalid input. Please enter a number.
Choose option: 3
Balance: 1200
Choose option: 4
Enter withdrawal amount: 400
Withdrawals possible: 3
Choose option: 3
Balance: 1200
Choose option: 5
Goodbye!
```